THEME 3B · HACKTOBERFEST HACK DAY · 24 OCT 2026

Receipt to Expense.

Snap a receipt. Our fine-tuned Gemma 4 reads it, logs the expense and splits the bill with friends.

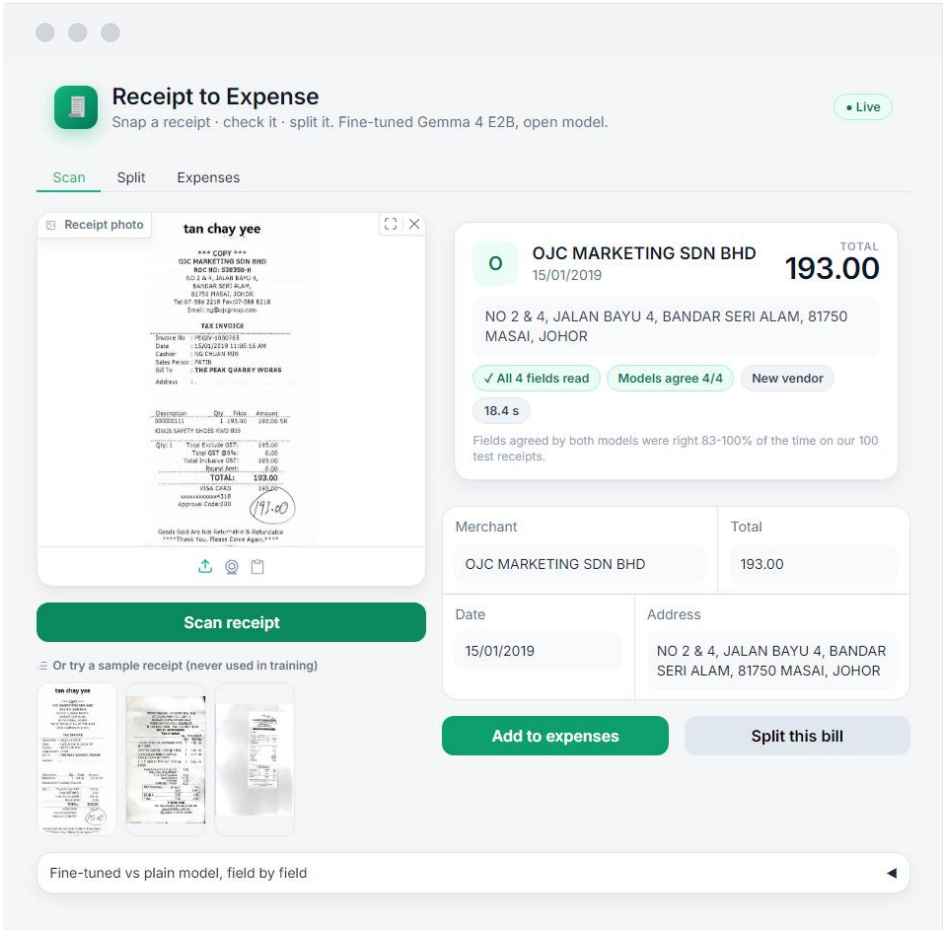
Team Come_Join_Qarc · SSIT Tumkur

Lead: Mohammed Owais

Members: Mohammed Muzakkir, Mohammad Tariq Afzal, Mohammed Ayaan, Mohammed Younus

10x

more receipts read perfectly than the base model



Students still type receipts in by hand.

Expense apps today	Receipt to Expense
Closed, paid, cloud OCR	Open model + one small adapter file
Built for company cards	One model from photo to fields, no OCR step
Never say which field to trust	Says what to double-check, then splits the bill

For hostel flats splitting groceries, clubs reimbursing fest bills and small shops logging purchases.

HOW IT WORKS

One open model, from photo to expense.

01

Snap

A phone photo goes straight to the model, image first, at high resolution. No OCR step.

02

Read

Gemma 4 E2B with our LoRA adapter, trained on public labelled receipts (SROIE) and tested on receipts it never saw.

03

Log and split

Edit the fields, add them to expenses, or split equally or by item; everyone settles up in the fewest payments.

What made the difference: **doubling image resolution beat doubling the model size.**

METHOD — HOW WE TRAINED IT

How we trained it.

01

Data

626 public receipts with labelled merchant, date, address and total (SROIE). 100 test receipts kept aside, never trained on.

02

Model

Gemma 4 E2B, an open 2B-class vision-language model. The receipt image goes first, then a one-line instruction.

03

LoRA fine-tune

Small rank-32 adapters on the vision and language layers: about 1% of the weights. One pass over the data, minutes on one GPU (Unsloth).

04

Check

A pixel-level check caught the trainer silently shrinking receipts to 512 px. Fixed: training now sees exactly what the app sees.

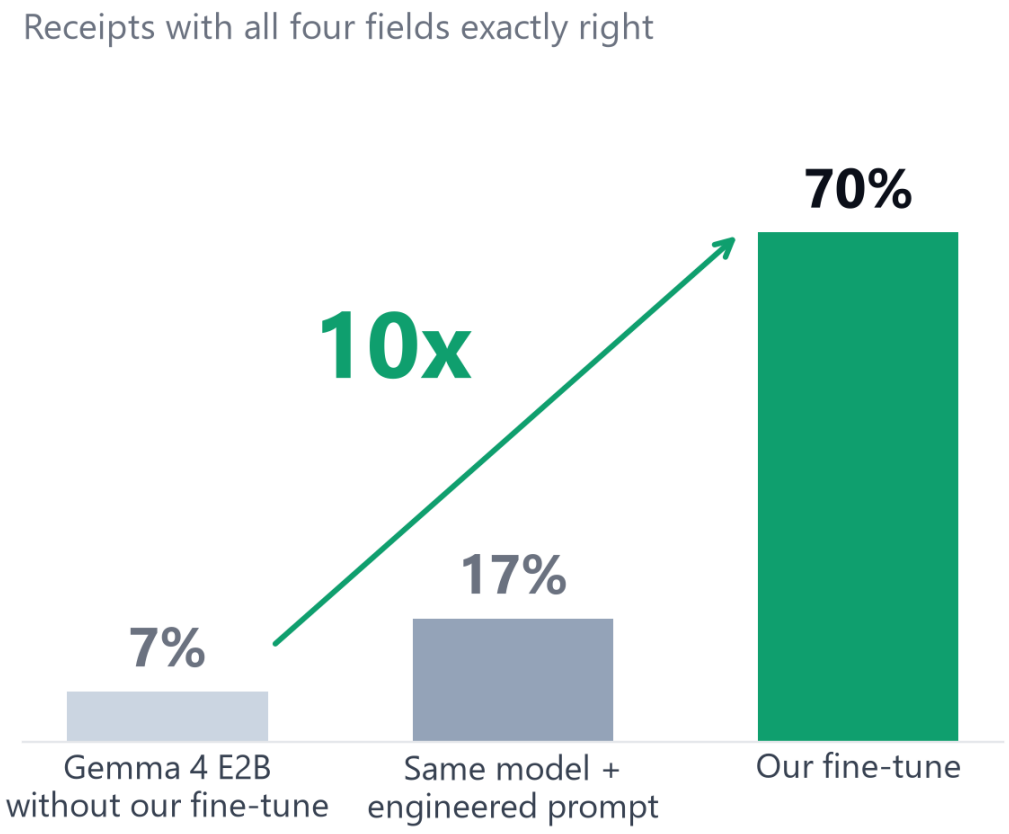
05

Sharpen

Doubled the image resolution (1120 visual tokens): the biggest single gain. Every model compared on the same settings.

Output: **one JSON line with merchant, date, address and total.** Greedy decoding, so the same photo always gives the same answer.

10x more receipts read perfectly.



Same Gemma 4 model, same settings, same 100 receipts it never saw.
Without our fine-tune: 7 in 100 read perfectly.
With it: 70 in 100.

100 held-out receipts · McNemar p < 1e-10

Fine-tuned version shown: best of 5 configurations, same test set.

3/4

fewer reading errors.

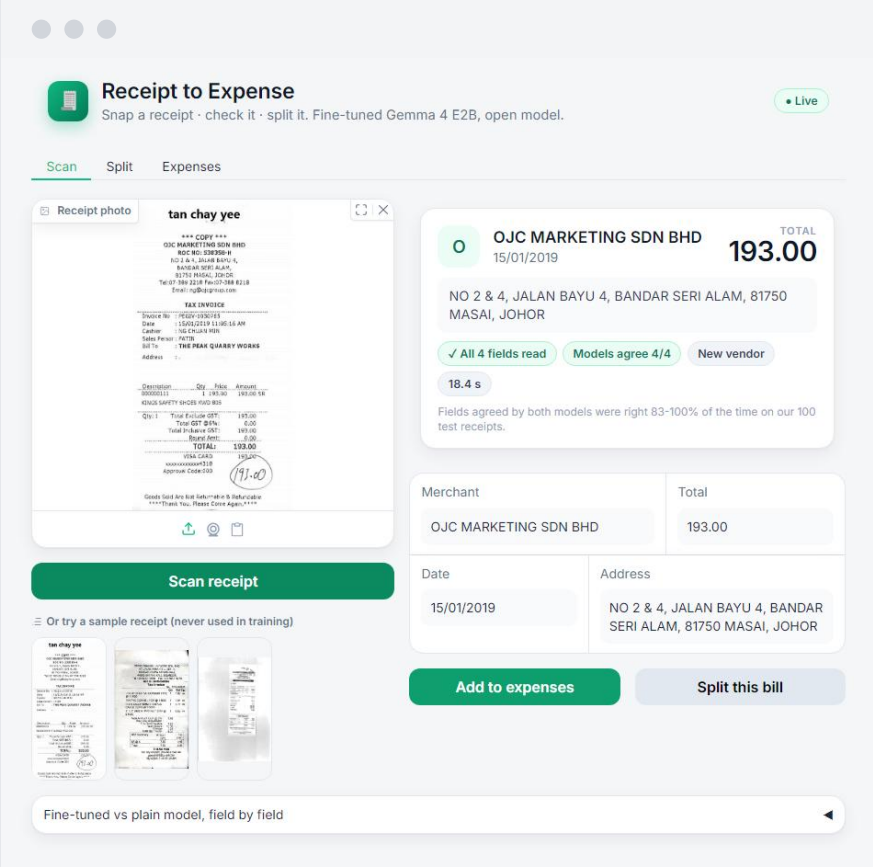
Field accuracy goes from 70% to 92% on the same model. Totals are right 98% of the time.

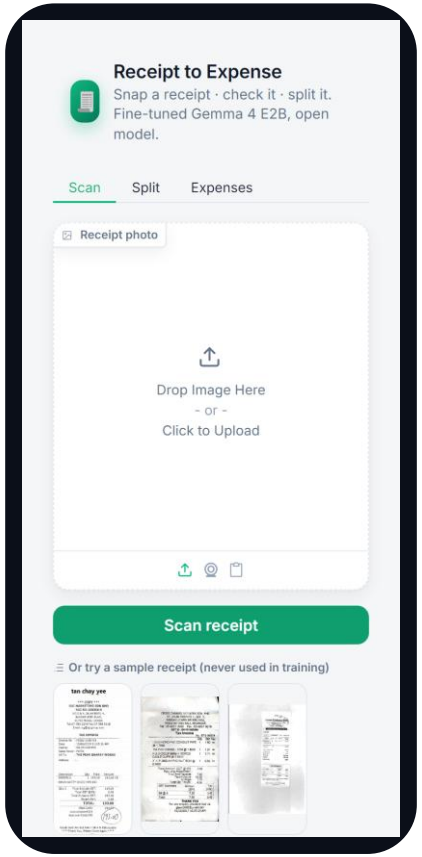
2x

better than prompt engineering.

A carefully engineered prompt gets 37% of receipts fully right. Our fine-tune gets 76%.

Snap it. Check it. Split it.





Judges: lend us a receipt.

Scan the code to try it on your phone.

f6753b53a276c78344.gradio.live

Trust check on every field
Split equally or by item
Expenses to CSV

On the projector: the polished UI. On phones: the public link, same fine-tuned model.

DELIVERED

Everything required, plus every bonus.

Required

- ✓ Fine-tuned model + training settings
- ✓ Dataset description
- ✓ Plain vs fine-tuned comparison
- ✓ Working app, live
- ✓ Where it helps, limits and why

Helps, limits, why: learns receipt layouts fast; next, Indian receipts.

Bonus

- ✓ Prompted baseline
- ✓ Training experiments
- ✓ Overfitting check
- ✓ Model size comparison (E4B)
- ✓ Combined with retrieval (RAG)
- ✓ Published on Hugging Face
- ✓ Error analysis
- ✓ Line items

Model card

huggingface.co/owaisnoe/gemma4-e2b-sroie-lora

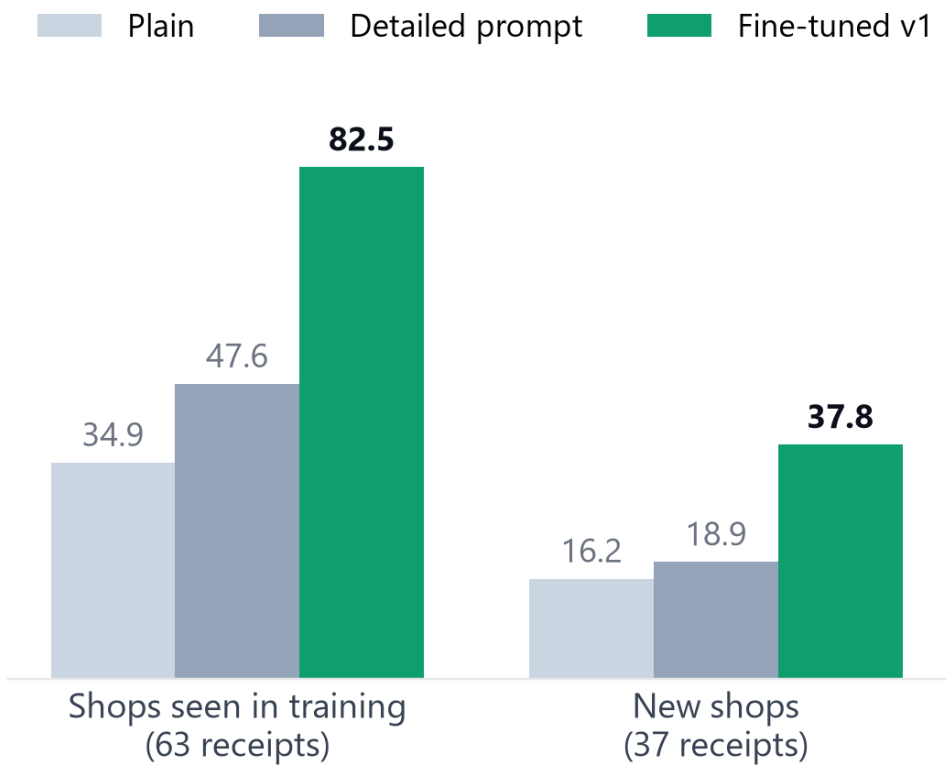
Next

Indian receipts

UPI requests from the split

On-device: receipts stay on the phone

Robustness.



Seen vs new shops (first model, budget 560): 63 of the 100 test receipts come from shops that also appear in training; on brand-new shops the fine-tune still doubles the prompt (37.8% vs 18.9%).

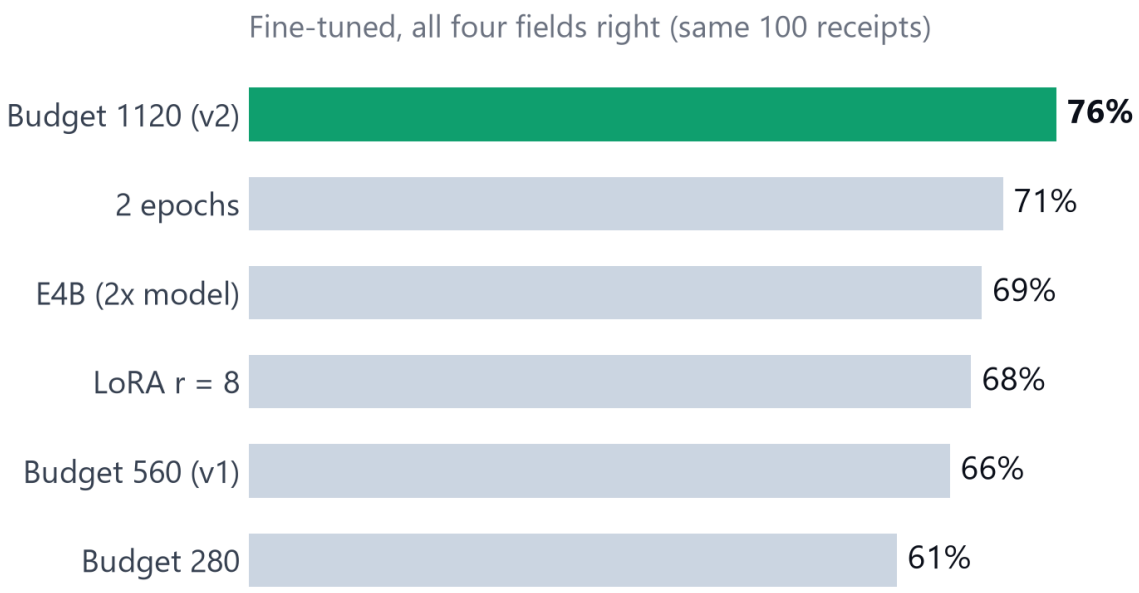
Other receipt styles (never trained on): WildReceipt, all labelled fields 58% plain vs 56% fine-tuned (not significant); CORD totals 87.5% vs 91.7% (not significant). No significant loss.

Retrieval (vendor memory from training labels): fine-tuned 66% to 69%; prompt 37% to 47%.

Line items, zero-shot on CORD: item F1 71% plain vs 79% fine-tuned.

APPENDIX — FOR QUESTIONS

Experiments and one silent bug.



Each arm changes one setting, on the same 100 receipts. Only resolution was significant: 1120 vs 560 tokens, 11 receipts better and 1 worse ($p = 0.006$). Doubling the model (E4B) gave 69%.

The collator bug: Unsloth's default collator shrank every receipt to 512 px and snapped it to 32 px, with the same token count but different pixels (max difference 0.68). Fixed with images as a dataset column and resize="max" (difference 0.0000).

Trust check: when fine-tuned and plain agree on a field, it was right 83-100% of the time (first model).

Data and settings.

Data	SROIE (ICDAR 2019, CC BY 4.0): 626 training receipts; 100 fixed test receipts (seed 0) from the 361-receipt test split; none trained on
Base model	unsloth/gemma-4-E2B-it, 16-bit LoRA adapters (never merged); r = 32, vision + language layers, 1.15% of weights
Training	lr 2e-4, cosine, adamw_8bit, batch 1 x grad-accum 4, 5 smoke + 157 steps (1 epoch), 2.5 min on a Colab G4
Versions	v1: 560 visual tokens (66% receipts, 89.5% fields) · v2: 1120 visual tokens (76% / 92%, best of 5 configurations)
Evaluation	image-first prompt, greedy decoding, batch size 1; strict + normalized scoring; exact McNemar tests
Baselines	plain model, same settings; engineered prompt measured at 560 tokens (at 1120 the plain model scores lower)
Compute	Colab G4 for speed; E2B LoRA needs about 8-10 GB per Unsloth's docs, so it should fit a free T4 (not tested)